



TED UNIVERSITY
Faculty of Engineering

Department of Computer Engineering

CMPE 492 Senior Project

ARIS

Final Report

Spring 2026

Taha Akdemir | ID: 12577008730

Ecem Nur Bilgi | ID: 14830754170

Şevkiye Sıla Kahya | ID: 14620089482

Elif Zeynep Elverişli | ID: 13942218426

Table of Contents

1. Introduction	3
2. Architecture and Design.....	4
2.1 System Architecture	4
2.2 Subsystem Decomposition	5
2.2.1 Input Processing Subsystem.....	5
2.2.2 Recognition and Interpretation Subsystem	6
2.2.3 Command Validation and Mapping Subsystem	7
2.2.4 Communication Subsystem.....	7
2.2.5 Backend and Monitoring Subsystem.....	8
2.2.6 User Interface Subsystem.....	8
2.3 Communication Between Subsystems	9
2.4 Design Improvements.....	11
3. Engineering Impact.....	12
3.1. Global and Societal Impact	12
3.2. Economic Impact.....	13
3.3. Environmental Impact	14
4. Contemporary Issues	14
4.1 Artificial Intelligence Ethics	14
4.2 Privacy and Security	15
4.3 Accessibility and Human-Computer Interaction.....	15
4.4 Reliability and Hallucination Risks in AI Systems.....	16
5. New Tools and Technologies	16
5.1 Raspberry Pi	16
5.2 Python.....	17
5.3 OpenCV	18
5.4 FastAPI.....	19
5.5 Web Dashboard	20
5.6 AI Models	21
6. Utilization of Library and Internet Resources	22
6.1 Documentation and Online Resources	22

6.2 New Tools and Technologies Used.....	23
7. Test Strategy.....	24
7.1 Tools and Setup.....	25
8. Test Results.....	25
9. Conclusion.....	27
10. References	29

1. Introduction

This final report presents the design, implementation, testing and evaluation phase of the ARIS (Artificial Intelligence-Assisted Responsive Interaction System) project. ARIS is an academic senior design project developed at TED University's Department of Computer Engineering to address the communication challenges faced in military and other high risk operational environment. In situations where user must remain fully focused on their surroundings, traditional handheld communication devices can reduce mobility, increase reaction time and also create additional safety risk. ARIS was developed to overcome these limitations by providing a hands-free communication system that interprets facial expressions, hand gestures and voice command in real time through intelligent helmet based platform.

In many operational scenarios, users need to communicate quickly and also securely without manually interacting with radios or other equipments. Environmental factors such as background noise, poor lighting and unstable network conditions can further complicate these processes. ARIS addresses these challenges by combining computer vision, speech recognition and also machine learning techniques with in modular and distributed system architecture. The helmet receives multiple modalities via an integrated camera and microphone. These multimodal inputs are then processed using lightweight artificial intelligence algorithms. These validated commands are then transmitted securely to central server, where they can be monitored through web based dashboard that provides real time status updates, telemetry data and also communication logs.

The significance of ARIS extend beyond its immediate prototype implementation. The project demonstrates how artificial intelligence, embedded systems and secure software architectures can be integrated to create practical solution for time critical environments. By focusing on real time performance, modularity, privacy and reliability, the ARIS project is prototype model of future wearable communication systems that could be extended to other industries such as military, rescue operations, and also industries that require safety standards. This report evaluates the completed system from software engineering perspective and examines its technical design decisions, implementation details, testing results and also broader engineering impact in terms of like social, economic, ethical and technological considerations.

2. Architecture and Design

2.1 System Architecture

ARIS functions as a distributed AI-supported smart helmet communication system which uses edge processing and secure communication methods and central system control for its operation. The system architecture enables user communication through multiple input methods which include hand gestures and facial expressions and voice commands without the need for users to touch physical communication devices. The system architecture of ARIS achieves its design objectives for high-risk environments through its requirements for quick system operation and modular system design and system reliability and secure data transfer.

The final architecture of ARIS follows a client-server structure. The helmet unit acts as the client-side edge device, while the backend server and web dashboard form the central coordination and monitoring layer. The helmet unit uses its camera and microphone modules for capture user inputs, which it processes through its local system to identify potential commands before it sends confirmed commands to backend. The backend server receives the command data, manages communication sessions, stores system-level logs, and provides real-time information to the web dashboard.

ARIS implements edge processing on the helmet unit as its main design choice for system operation. The system executes its first processing tasks on-site to identify video and audio material instead of transmitting complete video and audio recordings to an external server. The system framework reduces communication delays and maintains functionality during network instability while protecting user privacy through biometric data protection measures. The Raspberry Pi-based helmet unit therefore functions as an intelligent edge node which processes sensor data and conducts initial data handling and recognition tasks and performs command verification and communication setup functions.

The central server functions as system's main coordination point. It receives authenticated and validated command messages from helmet units, manages device sessions, stores event-level data, and supports the monitoring dashboard. Through this centralized structure, operators can see all connected helmets and their current activities and the system's operational state and all system

alerts and system logs. The design permits future expansion because new helmet units can join the existing backend system without any need to modify the current software design.

The web dashboard provides the main interface for command center operators and administrators. The system dashboard displays current system status through helmet connection status and command history and communication state and latency values and recognition confidence and alerts. The interface has a design which enables users to make quick decisions during high-pressure situations because of its easy-to-read features. Administrative users can monitor system logs and create user roles and track system operations without needing to touch the helmet devices.

The ARIS system architecture assigns edge-side processing tasks to edge devices while it delegates backend monitoring duties to remote systems. The helmet unit handles real-time recognition and communication tasks but the backend system and dashboard handle all coordination tasks and system monitoring needs and security functions and user management. The system separation enhances maintenance processes and enables teams to work on different system components and simplifies future system upgrades.

2.2 Subsystem Decomposition

The ARIS architecture uses multiple subsystems to achieve its design goal of operational system efficiency and system complexity reduction. The system pipeline has defined boundaries which each subsystem needs to operate within. The decomposition process improves system performance through better testing and debugging and system enhancement during the development phase.

2.2.1 Input Processing Subsystem

The Input Processing Subsystem is responsible for collecting visual and audio data from the helmet unit. The system receives visual frames through the camera and audio input through the microphone. The subsystem operates continuously until the helmet becomes inactive because it prepares data for recognition throughout its entire operational period.

The system processes visual input through facial expressions and hand gestures and other established visual patterns. The audio input includes spoken commands from the user. The system

needs basic data cleaning functions because input data quality determines recognition results. The system uses preprocessing techniques for visual input which involve preparing frames and changing their size and brightness and applying filters. The system applies preprocessing techniques to audio input through two procedures which involve reducing noise and identifying the time frame of important commands.

The subsystem uses low-level design to construct itself through components named CameraManager and MicrophoneManager and InputBuffer. The CameraManager controls camera initialization and frame capture, while the MicrophoneManager manages audio recording. The InputBuffer temporarily stores and synchronizes the captured data before sending it to the recognition layer. The system needs this buffering method because visual and audio data streams reach the system at different speeds which requires temporary storage until the system processes the data.

2.2.2 Recognition and Interpretation Subsystem

The Recognition and Interpretation Subsystem uses its preprocessed input data to determine what users want to achieve. The system includes distinct pathways which enable recognition of gestures and facial expressions and voice commands. The system manages recognition through its components which handle specific input types and generate recognized outputs with confidence assessment.

The gesture recognition component analyzes visual frames to detect predefined hand or body movements. The facial expression recognition component interprets facial patterns that may represent silent commands or emergency signals. The voice command recognition component processes audio input and matches spoken phrases with predefined command patterns. The three recognition paths enable ARIS to function as a system which supports multiple modes of operation.

The multimodal design increases system reliability because communication is not limited to a single input type. For noisy environments that make voice recognition impossible users can still use gestures and facial expressions to establish communication. Users can still use voice commands when visual recognition fails due to insufficient lighting or obstruction. The system achieves greater adaptability to changing situations because of this redundant capability.

2.2.3 Command Validation and Mapping Subsystem

The Command Validation and Mapping Subsystem handles command processing after it recognizes a potential command. This subsystem prevents uncertain or incorrectly recognized inputs from being transmitted as valid commands. The system processes confidence scores while it validates command structure and checks whether the recognized input matches an established command.

The system will either reject the command or ask the user to repeat their input when the confidence score falls below the accepted level. The system selects the most reliable match when multiple matches exist or it marks the result as uncertain. The validation process becomes essential because an incorrect command will create dangerous situations and confusion during actual operations.

The CommandMapper transforms the validated command candidate into a structured command message. The message contains data about command type, timestamp, device identifier, session information, priority level, and recognition confidence. The backend system can more easily transmit, log, and process commands.

2.2.4 Communication Subsystem

The Communication Subsystem functions as the data exchange control system which connects the helmet unit to the backend server. This system transmits through secure channels all validated commands and telemetry data and feedback messages and connection status information.

The subsystem enables users to exchange information through two distinct communication methods. The helmet can send commands and status data to the backend, while the backend can send acknowledgements, warnings, or instructions back to the helmet. This interaction allows the user to receive feedback after sending a command and helps the command center maintain awareness of system status.

The subsystem does additional work which requires it to resolve all connection problems. The communication layer provides buffering and retry capabilities because ARIS operates in areas with unstable network connectivity. Local message storage enables transmission of stored messages after the connection restores. The method prevents data loss while enhancing system dependability.

The low-level design of this subsystem contains three main classes which consist of CommunicationManager and ConnectionHandler and MessageBuffer. The

CommunicationManager coordinates incoming and outgoing messages, the ConnectionHandler manages network connectivity and reconnection attempts, and the MessageBuffer stores messages during communication interruption

2.2.5 Backend and Monitoring Subsystem

Backend and Monitoring Subsystem handles all server-side activities in its coordination work. The system handles command messages and telemetry data from helmet units while it controls session management and log storage and dashboard data display. The ARIS system uses this subsystem as its main control system.

The backend processes received data and makes it available to authorized users through the web interface. The system collects information about active devices and connection health and command transmission status and latency values and alerts and error events to support its monitoring functions. The data enables operators to assess system status while they handle emergency situations.

The backend system enables administrators to manage user roles and update system settings and control user access. Root users or administrators can manage system permissions and monitor operational records.

2.2.6 User Interface Subsystem

The User Interface Subsystem helps users control system functions by providing them with a web-based dashboard system. The system enables authorized users to access information about linked helmet devices, watch command activities, check connection states, and receive instant notification alerts. The dashboard uses a straightforward design which enables users to understand information quickly during critical moments.

The interface shows system information through readable tables which display data elements together with indicators and icons and alert panels. Operators can see critical warnings because they appear in a way which captures their attention. The dashboard provides users with both past logs and present system usage information which enables them to study earlier commands and faults and communication events.

The low-level design presents user interface elements through three components which include WebInterfaceController and RoleManager and LogManager. The WebInterfaceController controls dashboard rendering and updates. The RoleManager system enables the management of user access rights together with their respective roles. The LogManager system handles the documentation and retrieval processes of system events which serve monitoring and diagnostic and auditing purposes.

2.3 Communication Between Subsystems

ARIS establishes a communication system which operates through multiple processing stages. The first stage delivers data to each system which executes its task before sending results to the subsequent stage. The system enables users to comprehend data movement while enabling testing of individual components.

The process starts when the camera and microphone gather visual and audio information from the user. The Input Processing Subsystem prepares these data and sends synchronized input packages to the Recognition and Interpretation Subsystem. The recognition components analyze the input and generate possible command candidates with confidence scores. The Command Validation and Mapping Subsystem receives these command candidates for evaluation.

Valid command candidates undergo conversion into structured command messages. The Communication Subsystem then encrypts and transmits the message to the backend server. The backend system records the command it received while it updates its logs and monitoring data. The web dashboard retrieves or receives this information and displays it to authorized users.

The system handles general data flow through the following sequence of events:

Camera / Microphone → Input Processing → Recognition and Interpretation → Command Validation and Mapping → Communication Module → Backend Server → Web Dashboard

The helmet-side components mainly operate through local internal communication methods. The helmet unit executes input recognition and command processing tasks to achieve minimal delay. The helmet system establishes secure network connections to communicate with the backend

server. The system transmits command messages and telemetry information through structured data formats which mainly use JSON for their request and response body formats.

The system implements REST API communication which enables users to submit commands and receive telemetry updates and retrieve logs and manage configurations and perform authentication tasks. REST-based communication is suitable for structured, discrete operations because commands and status updates are generated as specific events. The API structure also improves maintainability with providing clear endpoints for different system functions.

WebSocket-based communication enables real-time monitoring and dashboard updates which need low-latency updates. WebSocket communication is especially valuable for delivering alerts and displaying live status changes and providing feedback on command execution. The system achieves simplicity through its REST-based operations while it maintains responsive performance by handling real-time streaming activities.

The communication between subsystems requires security measures. The system requires both helmet devices and users to complete authentication before they can start exchanging data. Dashboard functions become restricted through Role-Based Access Control which limits access based on user roles. The system uses encrypted communication channels to secure command and telemetry data. The system prohibits permanent storage and unnecessary transmission of sensitive raw input data which includes raw video and audio streams. The system collects operational metadata along with timestamps and confidence scores and event logs which contain command-level data.

The system provides error recovery solutions for all its subsystem communication functions. The system prevents users from sending commands when recognition confidence reaches an insufficient level while users receive guidance to repeat their input. The Communication Subsystem saves messages when the connection breaks and tries to send them again after the connection restores. The system produces a structured error response when the backend fails to handle a request. These mechanisms help ARIS remain stable.

2.4 Design Improvements

The development process of ARIS design evolved through three stages which made it into a more organized and modular and realistic system. The project development began with an idea to create a smart helmet which would identify gestures and facial expressions and voice commands. The design process evolved this initial concept into a complete software system which includes active monitoring mechanisms and security features and detailed system functions and monitoring capabilities.

The edge processing functions received their essential functions which were previously unclear between different systems. The initial design of ARIS described the system as a smart helmet communication system which processes multimodal inputs and sends out the results. The team selected the helmet-side edge device as the location to complete recognition and preprocessing tasks in the final design. The system now handles biometric data through processing on the helmet device which operates independently from the network

The system obtained its first major update through the creation of a modular subsystem structure. The system consisted of different components which handled input processing and recognition and command validation and communication and backend monitoring and security and user interface tasks. The system provides modularity which enables development and testing of each component as an independent unit. The system enables future development through its capacity to integrate new recognition systems and communication methods and dashboard functionalities without needing a complete system redesign.

The team made enhancements to the system through their work on communication design. The system now supports structured command messages and secure transmission and session handling and buffering during connection loss. The design includes reconnection and offline buffering mechanisms because the system does not depend on permanent network accessibility. The system becomes more dependable because of this feature which enables its operation in changing situations.

Security and privacy developed into enhanced conditions through design work. The design process required implementation of authentication systems, authorization protocols, encryption methods, and role-based access control as fundamental design components. The system treats raw video and

audio data as sensitive information and avoids permanent storage of these data during regular operation. The system maintains event-level data which includes essential details about command type and timestamp and transmission status and confidence score and error information. The privacy-preserving method decreases security risks and ethical concern

The user interface developed from its initial monitoring function into a complete web-based dashboard which provides administrative control capabilities. The dashboard enables users to monitor connected helmets in real time through its display of logs and alerts and connection status and system health information. The system enables role-based access control which permits administrators to create user permissions while tracking system activities in a secure manner.

The low-level design established precise class interfaces together with system package structure. The system development path becomes more understandable through the use of CameraManager, MicrophoneManager, InputBuffer, GestureRecognizer, VoiceCommandRecognizer, CommandValidator, CommandMapper, CommunicationManager, AuthenticationManager, and WebInterfaceController components. The object-level design enables better tracking of system requirements because it connects system requirements with architectural design and actual system implementation.

The development process created new features which transformed ARIS from its original prototype into a detailed engineering design that can be maintained. The final architecture enables recognition to occur in real time while providing secure communication and system monitoring capabilities and future system extension options which operate within project hardware and privacy restrictions.

3. Engineering Impact

3.1. Global and Societal Impact

ARIS has the potential to improve communication and situational awareness in environment where fast and reliable interaction is critical. Although the project was initially inspired by military

communication scenarios, the underlying technology can also be adapted to civilian domains such as emergency response, disaster management, industrial safety and also search and rescue operations. In these settings, user often need to send commands or alerts without interrupting their physical activities or diverting their attention from potentially dangerous surroundings. By enabling hands free communication through gestures, facial expressions, and voice commands, ARIS demonstrate how artificial intelligence can support safer and more efficient human machine interaction.

From societal perspective, ARIS represents technological advancement toward the greater trend of developing wearable and assistive devices that improve the capabilities of humans but don't attempt to extort their decisions. The system is designed to aid users by reducing the amount of manual work and reducing response times, while leaving the final decisions to humans. This approach align with responsible engineering principles and emphasizes the role of technology as a supportive tool. Moreover, the project reflects the increased significance of interdisciplinary approach in order to solve practical problems through combining the mentioned disciplines such as embedded systems, artificial intelligence, cybersecurity, software engineering. By exploring these technologies in an academic context, ARIS also helps prepare future engineers to design innovative system with meaningful social impact.

3.2. Economic Impact

The ARIS system has been designed as an economically feasible prototype, which uses readily available and low cost hardware components such as Raspberry Pi, off-the-shelf camera sensors, microphones and also open source software platforms. This approach demonstrates that advanced communication systems can be prototyped without requiring access to expensive military grade equipment. As a result, the project lowers the barrier to experimentation and provide practical foundation for future research and development in wearable communication technologies.

Within an economic perspective, ARIS based systems would lower expenses by increasing efficiency and reducing communication delays in high risk environments. Faster and more reliable communication may help organization respond more effectively to emergencies, reduce human error and also minimize downtime in critical operations. The modular design of ARIS also supports future commercialization by allowing individual subsystems, such as recognition modules or

monitoring dashboards. This is for being reused and adapted for different sectors. While large scale deployment would require additional investment in ruggedized hardware, testing and certification, the prototype demonstrates that sophisticated AI assisted communication systems can be developed using accessible and cost effective technologies.

3.3. Environmental Impact

The environmental impact of ARIS is primarily associated with its computational and electronic components. The helmet unit use embedded hardware to process visual and audio data locally, which consume electrical power during operation. The central server and web based monitoring dashboard also require computing resources to manage communication sessions, telemetry data and also system logs. In addition, the machine learning models used for gesture, facial expression and voice recognitions involve computational workloads that contribute to overall energy consumption.

At the same time, ARIS incorporate several design decisions that support sustainability. The edge processing capability helps in reducing the amount of data traffic by sending less bulky audiovisual content to distant cloud servers for processing. Unlike other systems, this one uses rechargeable batteries and has a modular design where individual parts can be substituted when they fail without having to discard the whole system. The adoption of open source software and commercial hardware promote recycling and makes the system durable. While this system has insignificant environmental impact, the project show how sustainable systems can be designed to meet high levels of performance.

4. Contemporary Issues

4.1 Artificial Intelligence Ethics

Artificial intelligence systems that use human behavior can cause ethical problems when they produce incorrect or misleading information. Since ARIS interprets with voice, facial expressions and hand gestures, system must process user input reliably and carefully. Some factors such as environmental noise, poor lighting or unclear gestures may affect the accuracy of recognition and lead to incorrect command generation.

To decrease the effect of these risks, ARIS uses predefined command library and confidence-based validation mechanisms instead of AI-generated outputs. Commands that do not meet validation requirements can be rejected before being transmitted to the next processing step. This helps reduce the possibility of unintended behavior.

In addition, ARIS is designed as a supportive communication system rather than autonomous decision-making platform. So, human control and oversight remain important during system operation in order to support responsible and transparent AI usage.

4.2 Privacy and Security

Both privacy and security are critical concerns in wearable AI systems because these systems process sensitive and personal user data. Since ARIS works with multimodal user input which includes face and voice, protecting the transmitted and stored data is considered as an important part of system design.

One of the major considerations in ARIS is unauthorized access to personal or operational information. To prevent these, ARIS is using secure communication mechanisms and authenticated system access. In addition, encrypted communication channels and access control based on role help to protect system data and give permission only authorized user access.

Additionally, the system was designed to be privacy-conscious. Rather than storing raw data permanently, ARIS relies on processed command-level data. This approach can minimize privacy risks without compromising system monitoring and operation.

4.3 Accessibility and Human-Computer Interaction

Accessibility and ease of use by humans are other important factors for wearable communication systems. ARIS is designed to support hands-free communication by allowing users to interact with the system through voice commands, gestures and facial expressions rather than traditional input methods.

Using multiple input types improves flexibility and usability under the various operational conditions. In ARIS, if one communication method becomes unavailable, another modality can still support the interaction with the system.

In addition, ARIS is designed to make user interaction more practical and intuitive during operation. The system helps reduce dependency on traditional input mechanisms such as keyboards or manual input by supporting alternative communication methods.

4.4 Reliability and Hallucination Risks in AI Systems

Environmental conditions and incomplete input data may affect recognition accuracy so reliability is an important issue in AI-based recognition systems. In ARIS, such factors as background noise, poor lighting conditions, unclear gestures or partial multimodal input may lead to incorrect interpretation of user intent.

Another consideration in AI systems is that the system incorrectly interprets patterns or generates unintended outputs. In such cases false recognition may result in incorrect command generation or action. To reduce the risk of these, ARIS uses predefined commands and command filtering procedures. The system is designed to accept only the commands that satisfy validation requirements before transmission process. In addition, using multiple input modalities helps improve reliability by supporting the recognition process through different communication channels.

5. New Tools and Technologies

5.1 Raspberry Pi

The ARIS helmet unit uses Raspberry Pi as its primary embedded processing system. The system functions as a computing platform which enables the collection of sensor data and the execution of preprocessing tasks and the operation of basic recognition systems and the establishment of backend server connections. The student-developed ARIS prototype system requires Raspberry Pi as its hardware solution because it offers affordable and accessible and portable and open-source

supported

alternatives.

The Raspberry Pi operates as an edge device within the ARIS system architecture. The system connects to the camera and microphone modules while processing local input data through its input handling capabilities. The helmet unit uses Raspberry Pi to process data which improves system performance during network outages and decreases waiting times. ARIS needs this capability because it requires instant command recognition but cannot rely entirely on cloud processing for its operations.

The system design of Raspberry Pi enables modular system development. The device allows developers to create and test separate camera and microphone and communication and software components. The platform enables development through Python and supports computer vision libraries and audio processing tools and network communication frameworks. This makes it suitable for implementing the main helmet-side components of ARIS, including input processing, recognition, command preparation, and telemetry transmission.

Raspberry Pi lacks the computational power of military-grade and high-performance embedded systems, yet it offers sufficient adaptability to create a proof-of-concept prototype. The design team selected lightweight models with optimized processing pipelines to maintain system responsiveness requirements. This trade-off supports the project's economic and technical constraints while still allowing real-time experimentation.

5.2 Python

Python functions as a primary software tool in ARIS because its capabilities enable efficient development of artificial intelligence systems and computer vision applications and audio processing solutions and backend software development. The language enables developers to build systems faster because its simplified design and extensive library collection.

Python enables helmet systems to handle camera operations and process video frames and conduct model inference and recognize commands and establish backend connections. Python scripts enable users to integrate OpenCV libraries together with machine learning frameworks. The recognition pipeline processes visual and audio inputs to produce command candidates which it

processes with high efficiency.

Python enables developers to build software systems using modular design techniques. Developers can build separate modules or classes that handle gesture recognition and facial expression recognition and voice command interpretation and command validation and communication handling. The system design follows ARIS's low-level structure which assigns specific tasks to each class.

Python supports edge processing functions while providing tools for backend API development and testing purposes. The software enables developers to build lightweight REST endpoints for command submission and telemetry updates and dashboard data retrieval and administrative operations by using FastAPI and other frameworks. Python enables developers to build AI processing systems for prototype development and complete system integration.

Python offers another benefit through its vast developer community and complete documentation resources. The team can concentrate on system behavior because Python development brings less complexity to their work on ARIS, which functions as an academic project. The system works well for embedded systems because its Raspberry Pi compatibility enables usage on that platform.

5.3 OpenCV

The ARIS system uses OpenCV to perform its computer vision functions. ARIS needs to capture and handle camera frames because it uses visual data to recognize user gestures and analyze their facial expressions. OpenCV provides all essential functions which enable users to obtain video frames and perform image processing and feature extraction and conduct visual analysis in real time.

OpenCV enables users to capture and process camera footage which will be used for recognition during the input processing phase. The standard preprocessing workflow includes multiple steps which involve resizing video frames and changing their color format and adjusting their brightness and contrast and removing background noise and extracting specific parts of the image. The operations between these two points help enhance input quality which researchers need to pass through to test their gesture and facial expression recognition models.

ARIS needs OpenCV because its functionality enables the system to process videos in real time. The visual pipeline should not introduce noticeable delays between the user's gesture or facial expression and the resulting command. OpenCV provides efficient image processing capabilities which fulfill this need when applied to lightweight models that run on Raspberry Pi devices.

OpenCV enables gesture recognition through two functions which include hand movement pattern detection and visual data generation for trained models. The system detects face regions from input images and normalizes these images while extracting essential frames needed for facial expression recognition. OpenCV serves as a visual recognition system which processes visual data before AI models execute their final recognition task.

OpenCV enables users to create enhancements which will benefit their projects in upcoming phases. The system can implement new visual capabilities through object detection and proximity detection and environmental awareness features which will be developed in future updates. OpenCV provides current recognition capabilities for ARIS while enabling future system development.

5.4 FastAPI

The ARIS prototype uses FastAPI as its modern backend framework which enables developers to create lightweight RESTful API services. The system architecture defines specific procedures for endpoint creation which include command data collection and telemetry update processing and helmet unit-backend web dashboard communication support.

FastAPI enables ARIS architecture to create operational endpoints which handle command submission and helmet status updates and log retrieval and authentication requests and configuration management tasks. The frontend dashboard enables users to access system information through API calls while helmet units transmit validated command messages to the backend via HTTP requests.

FastAPI offers users two primary benefits which include rapid performance and straightforward operation. The backend needs to deliver prompt service because ARIS requires its system to operate in real-time or near-real-time mode. FastAPI's asynchronous support enables better handling of multiple devices which connect to the system and users who access the dashboard. The system

needs this capability to handle future expansion when multiple helmet units transmit commands and telemetry data simultaneously.

FastAPI provides automatic API documentation through OpenAPI/Swagger. The functionality enables team members to test and develop software because they can access all available endpoints together with their required request formats and associated response structures. The existence of clear API documentation enhances system maintenance while simplifying the process of connecting frontend components with backend systems and helmet-side elements.

FastAPI usage supports the ARIS system's design which uses modular communication protocols. The system permits users to create distinct endpoints for each backend function which enables them to develop new services through the addition of extra endpoints. The upcoming versions will introduce new endpoints which will handle model updates and safety analytics and advanced logs and device-to-device communication support.

5.5 Web Dashboard

The ARIS system uses its web dashboard as its primary interface for system monitoring and administrative control. The system enables authorized personnel to monitor system operations through the helmet unit without needing to touch it. The dashboard displays active helmet status and command message details together with connection information and system alerts and log files and system health assessments.

Dashboard provides users with quick access to current operational situations. The ARIS system requires its interface to maintain high readability while offering straightforward design that users can quickly understand because it operates in environments with intense pressure. The system displays critical information through its interface which shows connection failures and command processing errors and urgent notifications and recognition results with low accuracy. Operators can use visual indicators which include colors and icons and tables and alert panels to quickly find out the present operational status of the system.

The dashboard uses API calls to connect with the backend while it establishes real-time communication links only when needed. REST API calls enable users to retrieve logs and device

information and control user access and view past data. WebSocket communication enables users to receive live alerts and watch real-time status updates while the dashboard shows instant updates

The dashboard also supports role-based access control. rent user groups to access its features. Users receive distinct access rights that correspond to their specific job functions. Standard helmet users can only access their personal device feedback whereas command center operators watch both active helmets and system logs and root users control recognition systems and user access rights and command dictionaries

The system provides better monitoring capabilities through its dashboard function which allows engineers to track system performance. The system allows the team to track command status through its functions which include command recognition and transmission and reception and logging and acknowledgment processes. The system provides debugging and maintenance support through its ability to display system errors and track historical events. The web dashboard functions as a user interface which provides essential operational and diagnostic capabilities to ARIS.

5.6 AI Models

The AI models help ARIS system work because they enable the system to understand user input through multiple modes of communication. The system uses recognition models to classify hand gestures, facial expressions, and predefined voice commands. The models create command candidates by processing raw sensor data and preprocessed sensor data which they use for validation and transmission purposes.

The gesture recognition model processes visual input from the camera and identifies predefined gestures. The gestures which the system recognizes correspond to specific commands which include moving forward and stopping and requesting support and sending an alert. The facial expression recognition model analyzes facial patterns that may be used for silent communication. This capability proves valuable in situations where the user needs to maintain silence or must not produce any sound. The voice command recognition model processes audio input and matches spoken phrases with predefined command templates.

Every AI recognition result comes with a specific confidence score. The confidence score indicates how certain the model is about its prediction. The system needs this value because it prevents automatic transmission of commands which the system considers uncertain. The Command

Validation Subsystem uses the confidence score to decide whether the command should be accepted, rejected, or repeated by the user.

The AI models used in ARIS are designed to be lightweight enough for edge deployment. The helmet unit processes recognition tasks according to which the models must function within Raspberry Pi's processing and memory boundaries. The system needs to find an equilibrium between achieving accurate model results and maintaining operational speed. The final design establishes its main focus on achieving fast and secure recognition performance under standard operational conditions through the use of small model-sized implementations that prevent operational delays and unnecessary resource utilization

The model structure also supports maintainability. The system allows users to update recognition models and command dictionaries without needing to rebuild the entire system. The team can use this system development to enhance recognition accuracy and create new commands and customize the system for various environments

AI-based recognition increases the flexibility of ARIS by allowing users to communicate through different modalities. The system has introduced new possibilities for use. Recognition performance depends on light conditions and background sounds and how users move and the camera position and the completeness of user input. ARIS uses validation together with confidence thresholds and fallback behavior and error handling to manage errors. The AI layer employs these mechanisms to enhance safety and establish reliability requirements which work well in critical communication systems.

6. Utilization of Library and Internet Resources

6.1 Documentation and Online Resources

Official documentation and online engineering resources were consulted throughout the design and implementation phases of ARIS. These resources supported system development, API integration, hardware configuration and debugging processes.

- **OpenCV Documentation:** Referenced for image processing operations, camera integration and real-time visual input handling.
- **MediaPipe Documentation:** Referenced for gesture tracking, facial feature detection and multimodal visual processing.
- **Flask Documentation:** Used during backend API development and client-server communication setup.
- **Raspberry Pi Documentation:** Referenced for hardware configuration, device setup and edge-side deployment processes.
- **MDN Web Docs:** Used during frontend and dashboard interface development using HTML, CSS and JavaScript technologies.
- **GitHub Documentation:** Referenced for repository management, version control workflows and collaborative software development practices.

6.2 New Tools and Technologies Used

During the implementation phase of ARIS, several modern tools and technologies were researched and integrated to support real-time multimodal communication, AI-based recognition, backend communication and web-based monitoring functionalities.

Edge and Embedded Technologies

- **Raspberry Pi 4:** Used as the edge computing device of the helmet unit for local processing, multimodal input acquisition and communication management.

AI and Computer Vision Technologies

- **OpenCV:** Utilized for image processing, visual preprocessing, distance estimation and camera-based data handling.
- **MediaPipe:** Used for hand gesture and facial landmark detection with lightweight real-time tracking capabilities suitable for edge hardware.

- **YOLO:** Integrated into the proximity detection subsystem for real-time object detection and safety analysis operations.

Backend and Communication Technologies

- **Flask:** Used for backend API development, telemetry processing and communication between system components.
- **WebSocket:** Enabled real-time communication between the backend and the monitoring dashboard for live system updates and alert synchronization.

Frontend Technologies

- **HTML, CSS, and JavaScript:** Used for developing the web-based monitoring dashboard and user interaction interface.

Development and Collaboration Tools

- **Python:** Selected as the primary programming language because of its support for AI, computer vision and rapid backend development.
- **Git and GitHub:** Used for version control, collaborative development and project management throughout the implementation process.

7. Test Strategy

Our test approach was mainly black box and integration based, meaning that the system was tested from the perspective of real user and operator rather than by directly inspecting internal code logic. Instead of only checking individual functions, we interacted with the system through its visible input and output, such as simulated helmet commands, telemetry data, backend API endpoint and also the web based monitoring dashboard.

The test process focused on verifying whether ARIS could correctly receive multimodal command data, transmit it to the backend, display real time updates on the dashboard and handle possible communication or input error. Since the physical helmet prototype and real battlefield conditions were outside the full testing scope, command recognition and telemetry behavior were tested through simulated data generators and controlled localhost execution.

The test plan covered the following five major components of the system:

1. Multimodal Command Recognition Flow
2. Helmet-to-Server Communication
3. Backend API Services
4. Real-Time Monitoring Dashboard
5. Authentication, Logging and Error Handling

Each component was tested for functionality, data correctness, response behavior, communication reliability, dashboard visibility and also error handling under both normal and edge case condition.

7.1 Tools and Setup

- **Framework / Backend:** FastAPI
- **Environment:** Localhost with simulated helmet devices and mock telemetry data
- **Test Style:** Black-box, integration-based, and behavior-driven testing
- **Automation Type:** Simulated command and telemetry transmission without direct internal state manipulation
- **Focus Areas:**
 - a. Command data integrity
 - b. Telemetry update correctness
 - c. API request and response validation
 - d. Real-time dashboard updates
 - e. Connection status handling
 - f. Error messages and recovery behavior
 - g. Backend integration with simulated helmet units

8. Test Results

We devised representative test cases covering all major components of the ARIS system. Since the project was developed as functional prototype, testing was primarily conducted in a controlled

development environment, generating telemetry data and predefined multimodal command inputs. The table shows selected test cases and their outcomes.

ID	Feature/Component	Test Case	Expected Outcome	Result	Notes
TC-RC-01	Multimodal Command Recognition	Submit predefined gesture, voice, and facial expression inputs	System correctly recognizes and maps the input to a valid command	Pass	Verified using simulated multimodal test data
TC-CM-01	Helmet-to-Server Communication	Transmit a validated command from the helmet unit	Command is securely received and processed by the backend server	Pass	Message stored successfully
TC-BE-01	Backend API Services	Send POST request to the /commands endpoint	API returns a successful response and stores the command data	Pass	Request and response structure validated
TC-DH-01	Real-Time Monitoring Dashboard	Generate telemetry and command updates	Dashboard displays current helmet status and incoming commands	Pass	WebSocket updates observed successfully
TC-AU-01	Authentication	Log in using valid operator credentials	Authorized access to the monitoring dashboard is granted	Pass	Session established correctly
TC-AC-01	Access Control	Attempt administrative action using a non-privileged account	Access is denied according to user role permissions	Pass	Role-based access control functioning correctly
TC-ER-01	Error Handling	Send malformed or incomplete command data	System rejects invalid input and returns an appropriate error message	Pass	Validation rules executed successfully
TC-CN-01	Connection Recovery	Simulate temporary network interruption	Commands are buffered and synchronized after reconnection	Pass	No data loss observed

TC-SEC-01	Secure Communication	Inspect transmitted command payload	Data is transmitted through encrypted communication channels	Pass	Secure transmission verified
TC-LOG-01	Logging Service	Execute commands and generate telemetry	Events are recorded with timestamps and displayed in logs	Pass	Logging functionality confirmed

The test results denote that ARIS prototype successfully performs its core functions including multimodal command recognition, secure communication, backend processing, real time monitoring and also access control. The system also demonstrated reliable behavior when handling invalid inputs and temporary connection failures.

Although testing was conducted primarily in a localhost environment with simulated devices rather than full field deployment, results confirm that the implemented architecture is stable and functionally complete.

9. Conclusion

ARIS is developed as an AI assisted helmet designed to support hands-free interaction in especially with high-risk operational environments. Project is combined with artificial intelligence, secure communication mechanisms, embedded systems and web-based monitoring technologies within a modular system architecture. By interpreting gestures, facial expressions, and voice commands in real time, ARIS aimed to reduce dependency on traditional handheld communication methods and improve operational efficiency.

Throughout the development process, the project evolved from a conceptual smart helmet idea into a distributed system architecture consisting of helmet-side edge processing, backend communication services, monitoring infrastructure, and secure subsystem interaction. The implementation process demonstrated the integration of multimodal recognition technologies, command validation mechanisms, communication protocols, and real-time monitoring functionalities within a unified prototype structure. In addition, the project addressed important engineering considerations such as privacy, reliability, accessibility, and secure communication.

The development of ARIS also provided practical experience with modern technologies related to artificial intelligence, computer vision, embedded systems, and real-time web communication. Although the project was implemented as an academic prototype, the resulting architecture supports future improvements such as advanced recognition models, expanded monitoring capabilities, improved communication reliability, and additional operational functionalities. Overall, ARIS demonstrated the feasibility of a real-time AI-assisted wearable communication system operating under practical engineering constraints.

10. References

- [1] OpenCV Documentation, “Open Source Computer Vision Library.” Available: <https://docs.opencv.org/>
- [2] MediaPipe Documentation, “Google MediaPipe Framework.” Available: <https://developers.google.com/mediapipe>
- [3] Raspberry Pi Documentation, “Raspberry Pi Official Documentation.” Available: <https://www.raspberrypi.com/documentation/>
- [4] FastAPI Documentation, “FastAPI Framework Documentation.” Available: <https://fastapi.tiangolo.com/>
- [5] MDN Web Docs, “HTML, CSS, and JavaScript Documentation.” Available: <https://developer.mozilla.org/>
- [6] GitHub Documentation, “GitHub Docs.” Available: <https://docs.github.com/>
- [7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [8] T. Baltrušaitis, C. Ahuja, and L.-P. Morency, “Multimodal Machine Learning: A Survey and Taxonomy,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, no. 2, pp. 423–443, 2019.
- [9] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [10] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, 2017.
- [11] OWASP Foundation, “OWASP Top Ten Web Application Security Risks.” Available: <https://owasp.org/www-project-top-ten/>
- [12] Python Software Foundation, “Python Documentation.” Available: <https://docs.python.org/3/>
- [13] WebSocket Protocol Documentation, “RFC 6455: The WebSocket Protocol.” Available: <https://datatracker.ietf.org/doc/html/rfc6455>